

中国大学生计算机设计大赛



物联网应用类作品技术文档

作品编号： 2026015123

作品名称： AquaGarden——基于多模态 AI 机械臂的居家实时调度水陆生态圈

作 者： 王凯杰，王大智，奚光磊，王妍睿

版本编号： v1.3

填写日期： 2026/4/25

填写说明：

- 1、本文档适用于物联网应用类作品；
- 2、正文一律用五号宋体，一级标题为二号黑体，其他级别标题如有需要，可根据需要设置；
- 3、本文档为简要文档，不宜长篇大论，简明扼要为上；
- 4、提交文档时，以 PDF 格式提交本文档；
- 5、本文档内容是正式参赛内容组成部分，务必真实填写；如不属实，将导致奖项等级降低甚至终止本作品参加比赛。

目录

第 1 章	作品概述	4
1.1	作品名称与应用领域	4
1.2	系统整体构成	4
1.3	鱼缸端功能	5
1.4	盆栽端功能	6
1.5	机械臂功能	7
1.6	软件平台功能	9
1.7	创新性说明	10
第 2 章	需求分析	11
2.1	开发背景	11
2.2	市场背景与政策环境	11
2.3	用户痛点分析	12
2.6	主要功能需求	15
2.7	主要性能指标	16
第 3 章	技术方案	16
3.1	方案概述	16
3.2	硬件方案	16
3.2.1	三层主控架构	17
3.2.2	传感器与执行器选型	18
3.2.3	机械臂与视觉模块	18
3.3	算法方案	18
3.3.1	鱼类行为监测	18
3.3.2	机械臂任务算法	19
3.3.3	多模态生态状态分析	19
3.4	软件方案	20
3.4.1	整体架构	20
3.4.2	后端与通信	20
3.4.3	前端控制台	20
第 4 章	方案实现	21
4.1	实现概述	21
4.2	硬件实现	21
4.2.1	外设层固件（RA6E2）	21
4.2.2	机械臂执行层（STM32）	21
4.2.3	规划层（树莓派 5 + ROS2）	21
4.3	算法实现	22
4.3.1	视觉模型部署	22
4.3.2	三项机械臂任务实现	22
4.3.3	大模型交互实现	23
4.4	软件实现	23
4.4.1	后端服务实现	23

4.4.2 前端控制台实现	23
4.4.3 机械臂 Agent 实现	24
第 5 章 测试报告	24
5.1 测试目的与环境	24
5.2 硬件测试	24
5.2.1 测试目的	24
5.2.2 测试对象与技术依据	24
5.2.3 测试环境	25
5.2.4 测试内容与方法	25
5.2.5 模块测试结果	26
5.2.6 FreeRTOS 联调测试结果	33
5.3 算法测试	34
5.3.1 视觉检测测试	34
5.3.2 机械臂任务成功率测试	35
5.4 软件测试	35
5.4.1 接口测试	35
5.4.2 前端交互测试	35
第 6 章 应用前景	35
6.1 居家消费市场	35
6.2 教育与科普场景	36
6.3 商业展示与公共空间场景	36
6.4 养老与康复场景	36
6.5 可持续发展与政策契合性	37

参考文献

第1章 作品概述

1.1 作品名称与应用领域

作品名称：AquaGarden——基于多模态 AI 机械臂的居家实时调度水陆生态圈

Aqua 取自 Aquarium（水族馆）的首部，Garden 对应盆栽花园，合在一起就是这套系统想做的事情：把一个鱼缸和一盆植物连成一套真正运转的生态。“水陆生态圈”不是比喻，而是字面意义上的水域+陆地两个子系统，通过水循环和机械臂把它们串在一起。

应用领域上，AquaGarden 属于智能家居与消费级物联网的交叉范畴，同时也带有一点城市微农业的影子。具体来说，系统面向有养鱼或养植物需求但又没有太多时间打理的城市居民，放在客厅、书房或阳台均可，不需要专业水产或园艺知识。此外，系统的鱼菜共生模式、多模态 AI 感知和机械臂自动执行等技术方向，在农业物联网与智能机器人两个领域同样有一定参考价值。

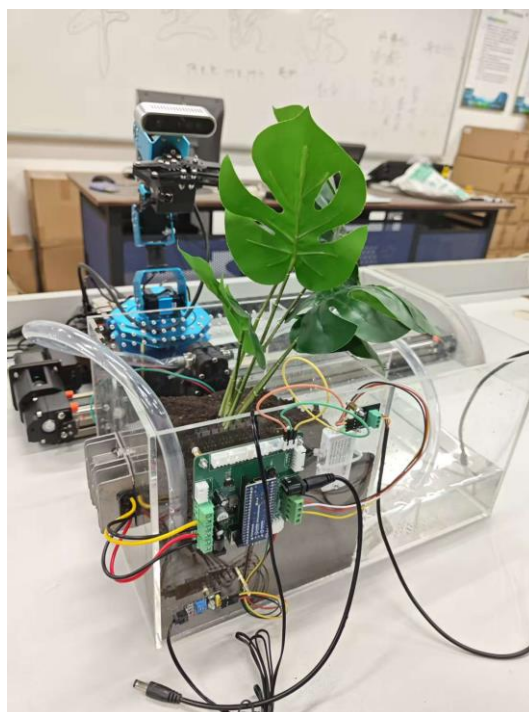


图 1-1 系统实物图

1.2 系统整体构成

系统由三个相互协作的子系统构成：鱼缸端、盆栽端和机械臂。外观上采用“一左一右”布局，左半部分为鱼缸，右半部分为种植土壤。机械臂安装在产品一侧的水平滑轨上，可沿滑轨在鱼缸区与盆栽区之间自动横移，从而用一台机械臂完成两个区域的所有操作任务，不占用中间空间，整体结构更紧凑。

硬件主控分三层：瑞萨 RA 系列单片机负责底层外设驱动，包括各路传感器采集、水泵、灯光、雾化模块的控制；STM32 专职驱动机械臂的六个舵机关节；树莓派运行 ROS 2[8]，承担机械臂运动规划、视觉算法推理和对外通信。三层之间通过串口和局域网通信，职责明确、互不耦合，某一层出问题不影响其他部分正常工作。软件侧以 Spring Boot 为后端、Vue 3

为前端，构建 Web 控制台；机械臂 Agent 部分用 Python FastAPI 实现，通过 WebSocket 与前端实时交互。

各主要硬件模块及其职责如表 1-1 所示。

表 1-1 主要硬件与软件模块一览

模块	型号/框架	所在层级	主要职责
外设主控	瑞萨 RA 系列单片机	外设层	传感器采集、水泵/灯光/雾化驱动
机械臂主控	STM32	执行层	六路舵机 PWM 控制
上位机	树莓派 + ROS 2	规划层	运动规划、视觉推理、云端通信
深度相机	光鉴 Aurora 930	规划层	彩色图/深度图/点云输出
延时摄影相机	RGB 摄像头（独立固定）	外设层	盆栽端延时摄影
后端服务	Spring Boot 3.3	软件层	数据存储、API、WebSocket 推送
前端	Vue 3 + Vite	软件层	数据可视化、远程控制界面
机械臂 Agent	Python FastAPI	软件层	LLM 对话桥接、任务调度

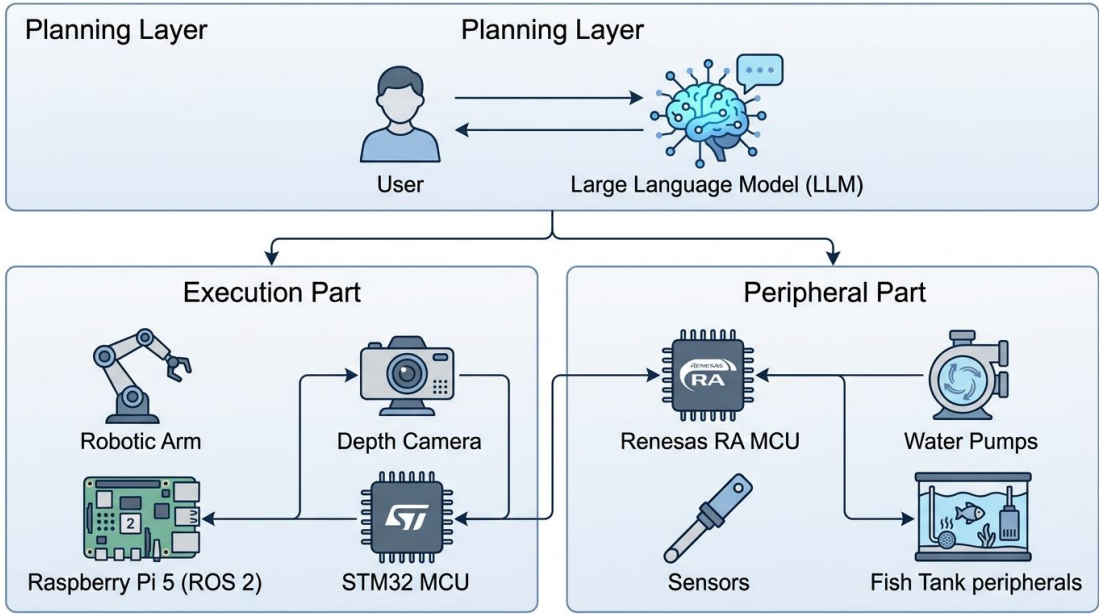


图 1-2 系统框架图概览

1.3 鱼缸端功能

水质检测是鱼缸端最基础的功能。系统集成了水温传感器、浑浊度传感器和溶解氧传感器共三路，数据每 5 秒采集一次并上传到后端。超过预设阈值时，系统自动向用户推送告警通知，同时根据告警类型触发对应的执行动作，比如水温过高时加快水循环以辅助散热[1]。

各水质传感器的主要参数如表 1-2 所示。

表 1-2 水质传感器参数一览

传感器	检测指标	量程	精度	告警阈值（参考）
水温传感器	水体温度	0~80 ℃	±0.5 ℃	<18 ℃ 或 >32 ℃

传感器	检测指标	量程	精度	告警阈值（参考）
浑浊度传感器	水体透明度	0~3000 NTU	±5 NTU	>500 NTU
溶解氧传感器	水中含氧量	0~20 mg/L	±0.3 mg/L	<5 mg/L

自动换水与水循环是整个系统鱼菜共生理念的物理实现。鱼缸底部的水泵将含有鱼类代谢产物（氨氮、磷等）的底层水抽出，经管道输送到盆栽端的灌溉系统，同时向鱼缸补入新水。整个过程完全自动化，用户不需要手动换水，也不需要把废水倒掉。鱼的粪便对植物来说是天然有机肥，这部分养分通过水循环被直接利用。

深度学习鱼类监测基于 YOLOv8[7] 实现。固定在鱼缸上方或侧面的摄像头实时采集画面，模型对缸内鱼类进行品种识别和运动轨迹跟踪。系统会持续分析鱼类的行为模式，比如是否长时间静止、是否出现翻肚、是否异常聚集在某个角落，结合水质数据综合判断鱼类的健康状态。一旦检测到异常，立即触发告警并记录当时的传感器数据和视频片段，方便用户事后复查[6]。

智能光照与音乐根据时段和鱼类昼夜节律自动调节灯光亮度和色温，夜间自动调暗。系统同时支持播放对鱼类有益的低频音乐，音量和曲目可以在 Web 端配置。

1.4 盆栽端功能

土壤监测方面，系统接入土壤湿度传感器和养分检测模块，实时监测含水率及氮磷钾等养分含量。数据上传后端后，系统根据阈值判断是否需要补水或追肥，并将建议推送给用户，或者直接触发自动灌溉动作。

自动灌溉的水源来自鱼缸端的循环废水。系统根据土壤湿度数据决定灌溉的时机和水量，不需要用户手动浇水。由于鱼缸废水本身含有营养成分，植物在大多数情况下不需要额外施化学肥料，降低了维护成本和化学品使用[2]。

智能补光与雾化加湿：系统配备光照传感器，当自然光不足时自动开启补光灯，亮度和开关时段可配置。雾化模块根据环境湿度数据控制开关，为植物维持适宜的空气湿度。这两个功能对大多数室内观赏植物来说有明显改善效果，尤其是在北方冬季或空调直吹的室内环境。

延时摄影功能由固定摄像头负责，每隔固定时间（默认 30 分钟）自动拍一张照片，后端将这些照片自动合成为延时视频，记录植物从小苗到成株的完整生长过程。这一功能对用户来说更多是情感价值，但也方便在竞赛展示中直观呈现系统的长期运行效果。

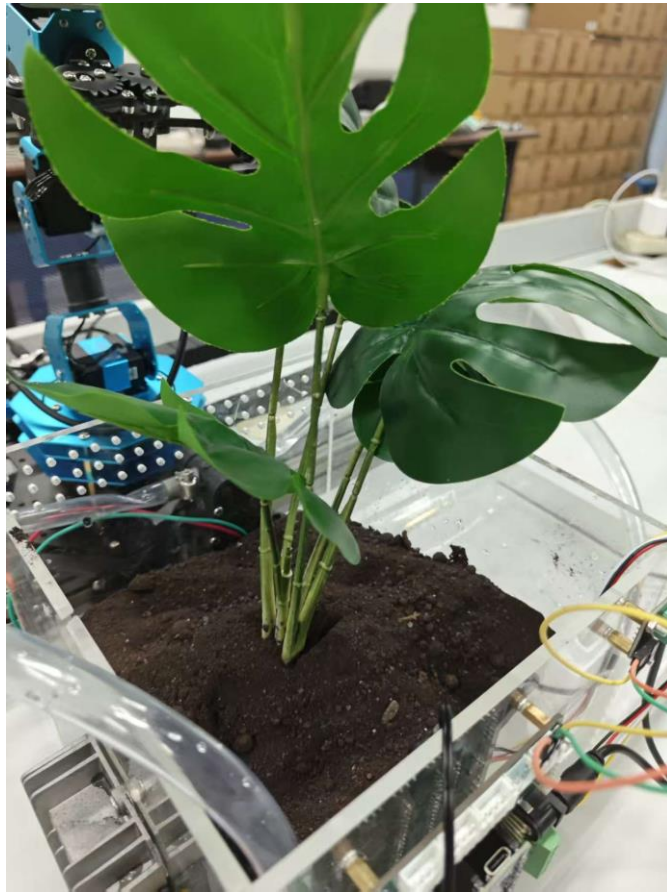


图 1-3 盆栽端展示

1.5 机械臂功能

机械臂是 AquaGarden 中技术难度最高、也是最有差异化价值的部分。臂体为六自由度结构，末端装有光鉴 Aurora 930 深度相机[10]。Aurora 930 内置彩色图像、深度图和点云三路输出，既能获取目标的精确三维坐标，也能直接提供 RGB 画面用于目标识别，无需额外搭配视觉相机。系统另外固定了一个独立 RGB 摄像头，专门用于盆栽端的延时摄影，与机械臂的操作视觉任务完全独立。



图 1-4 机械臂端展示

机械臂支持的任务类型如表 1-3 所示

表 1- 3 机械臂任务一览

任务名称	触发方式	是否依赖视觉	执行说明
叶片修剪	自动检测 / 语音指令	✓ Aurora 930 点云定位	YOLO 识别枯叶→3D 坐标→路径规划→剪刀触发
漂浮物清理	自动检测 / 语音指令	✓ 摄像头检测水面	识别漂浮物→定位→伸入水面捞取
松土	定时 / 语音指令	✗ 预设轨迹执行	按固定轨迹在土面插入动作
投食辅助	定时 / 语音指令	✗	摇动饲料罐或拨动水面
大模型交互动作	语音唤醒	✗	执行预录动作（打招呼、投食等）

叶片修剪是机械臂最复杂的任务。系统通过摄像头识别出植物中的枯黄叶片(基于 YOLOv8[7]语义分割,可识别颜色异常区域),然后用 Aurora 930 获取目标叶片在三维空间中的坐标,机械臂规划路径后移动到目标位置,末端工具触发剪刀完成剪枝。目前的实现方案中,叶片识别和三维定位已经可以自动完成,机械臂的最终剪切动作支持全自动和半自动(人工确认后执行)两种模式。

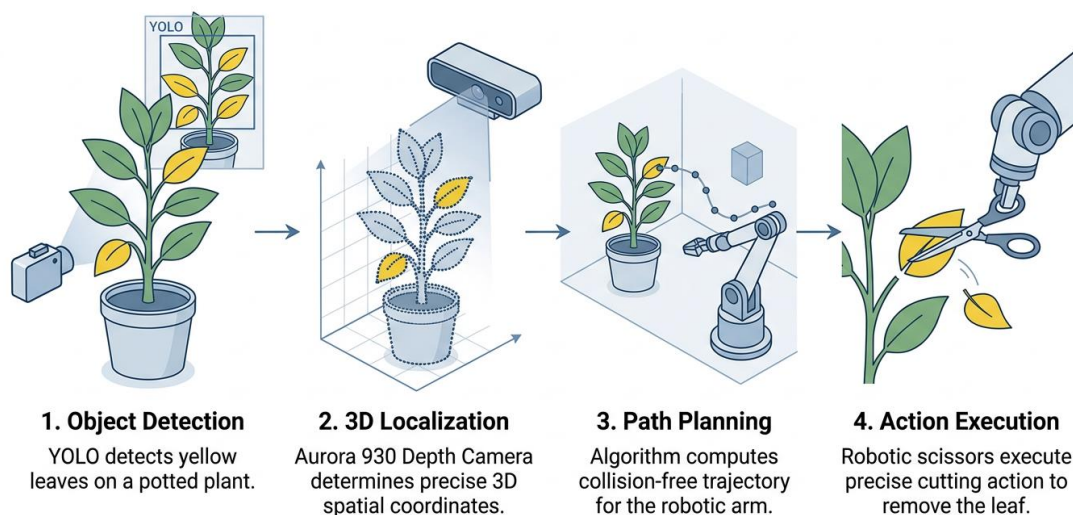


图 1-5 机械臂叶片修剪功能架构流程图

松土是机械臂最容易实现的任务之一。系统预先在盆栽坐标系中定义好松土轨迹，机械臂按固定路径在土面执行插入动作，不依赖实时视觉感知，稳定性最高。

语音唤醒与大模型交互：用户说唤醒词后，系统通过阿里云大模型（DashScope）解析自然语言指令，将意图映射到具体的机械臂任务并调度执行。系统也支持预录动作的回放，比如固定的“投食动作”或“打招呼动作”可以通过语音一句话触发

1.6 软件平台功能

Web 端基于 Spring Boot + Vue 3 构建，主要展示各传感器的实时数据和历史折线图，同时提供远程手动控制各执行器的接口（水泵开关、灯光亮度、机械臂复位等）。数据通过 WebSocket 推送，刷新延迟在 2 秒以内。

机械臂控制台以独立子页面呈现，显示当前机械臂姿态、任务队列和执行状态，支持用户手动介入取消当前任务或调整执行顺序。大模型对话窗口集成在 Web 端，用户可以直接在浏览器里询问“最近鱼的状态怎么样”，系统结合历史传感器数据给出自然语言分析报告，而不是单纯列出数字。

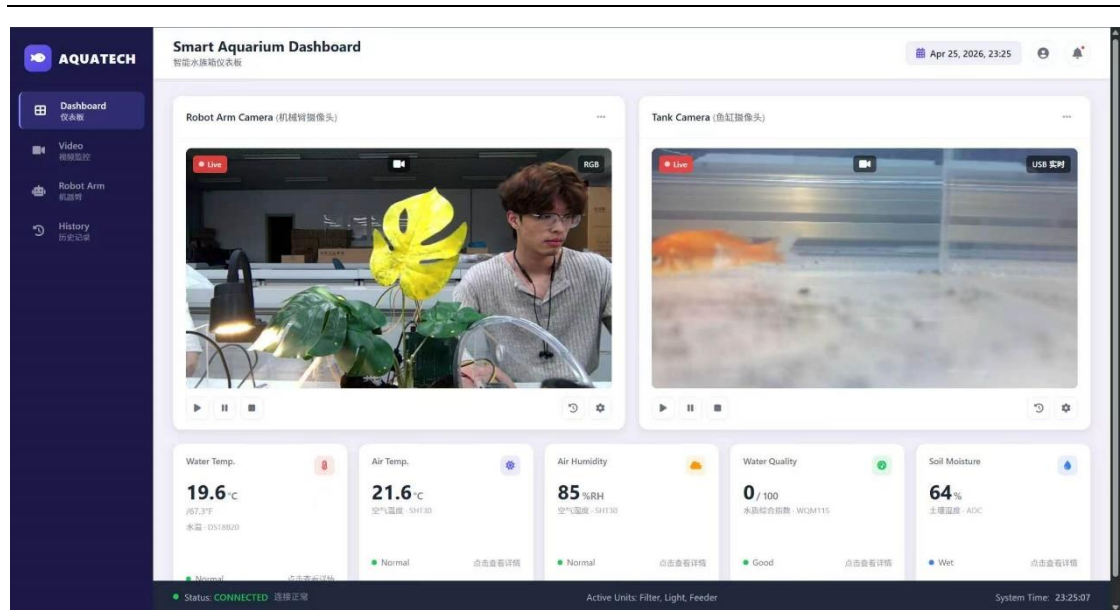


图 1-6 Web 端控制台界面截图（传感器数据面板+机械臂控制台）

1.7 创新性说明

设计思路创新：AquaGarden 的核心设计出发点是“鱼缸废水不应该被浪费”。市面上的智能鱼缸和智慧盆栽一直是两个独立的产品方向，从来没有人是一套居家产品里把两者在物理管路、数据和执行层面真正打通。我们的思路是：鱼缸端产生的富营养废水直接输送给盆栽端，盆栽端的植物根系进一步吸收水中剩余营养，同时一台机械臂在两个区域都能工作，形成“一臂多用、水陆共享”的整体调度模式。

技术创新：系统在感知层引入多模态融合机制，不只依赖传感器阈值判断，而是将水质时序数据、鱼类行为视频特征、环境传感数据同时送入大模型进行综合分析，输出比单一参数更全面的生态健康评估。在机械臂视觉引导方面，选用光鉴 Aurora 930[10] 作为末端视觉传感器，其内置的点云、深度和 RGB 三路输出使得目标识别与三维定位可以在同一设备上完成，省去了多相机标定的复杂度，数据对齐精度也更高。YOLO[7] 语义分割驱动机械臂精准剪枝的完整链路（识别→三维定位→路径规划→执行），目前在居家场景中尚无商业产品实现。

硬件创新：系统采用瑞萨 RA 单片机（外设层）+ STM32（执行层）+ 树莓派 ROS 2（规划层）的三层异构主控架构，三层职责清晰、解耦设计。外设层和执行层均为嵌入式实时系统，响应速度快；规划层运行完整的 ROS 2[8] 框架，可以方便地接入各类机器人算法库和传感器驱动。这种分层设计使得系统在调试和迭代时每层可以独立测试，不会因为某一层的问题拖累整体。在结构设计上，机械臂并非固定安装在两区之间，而是挂载在产品一侧的水平滑轨上，可由电机驱动自动横移，按需切换到鱼缸区或盆栽区上方再执行操作。这一设计使得同一台机械臂可以完整覆盖两个工作区域，避免了双臂方案的硬件冗余，也不需要在水缸和盆栽之间预留机械臂固定底座的空间，整体结构更紧凑，产品外观更简洁。

应用创新：从用户体验角度来看，AquaGarden 实现了“零手动换水”——用户不需要定期人工换水或手动浇灌盆栽，整套流程完全自动调度。同时，通过大模型赋能的自然语言交互，用户可以直接开口说话控制机械臂，不需要在 App 里一层层点菜单，这对不熟悉智能硬件的用户也相对友好。延时摄影、鱼类行为分析、植物生长记录等功能也让“养护”本身变得更有趣，不只是单纯的自动化工具。

四类创新点总结如表 1-4 所示。

表 1-4 创新性说明汇总

创新维度	创新点	对标现有方案的差距
设计思路	鱼菜共生闭环，废水变肥料，一臂双区服务	现有产品鱼缸/盆栽完全独立，废水直接丢弃
技术	多模态融合感知 + Aurora 930 一体化视觉定位 + YOLO 驱动机械臂剪枝	现有方案仅做传感器阈值报警，无视觉执行闭环
硬件	瑞萨 RA+STM32+树莓派三层异构解耦架构；机械臂挂载滑轨自动横移，单臂覆盖双区	现有产品多为单 MCU 方案，扩展性差；无产品用滑轨实现跨区机械臂调度
应用	零手动换水 + 自然语言控制机械臂	现有智能鱼缸仍需人工换水，App 操作繁琐

第2章 需求分析

2.1 开发背景

做这个项目，最初的触发点其实很日常。团队里有人在宿舍养了一缸鱼，断断续续养了将近一年，最大的感受是：养鱼这件事对时间规律的要求很高。水要定期换，pH 要隔几天测一次，喂食不能太随意，出去玩几天回来水可能已经发白发臭。和鱼一起死掉的，还有一盆没人浇水的绿萝。

把这两件事放在一起想了一下，发现问题的结构其实很像：两者都属于“需要规律性人工干预、但现有自动化工具很不完善”的类别。更有意思的是，鱼缸换出来的废水其实是很好的植物肥料——鱼类排泄物中的氨氮、磷等成分，正是植物生长所需的营养元素。大多数人换水都是直接倒掉，这是一个很典型的资源浪费。

这两个观察合在一起，就有了 AquaGarden 的基本想法：做一套能自动调度的水陆一体系统，把鱼缸的废水利用起来给植物浇水，同时用机械臂处理两侧都需要的体力活（喂食、修剪、清洁），让用户真正做到“买来放在那里，系统自己跑”。

2.2 市场背景与政策环境

从市场规模来看，国内水族设备行业市场规模已超 500 亿元，且受消费升级和宠物经济驱动，近年来持续增长。根据相关市场调研数据，智能化是目前水族设备升级最明确的方向，越来越多的消费者希望通过手机远程管理鱼缸状态，不再依赖每天人工检测。当贝、小米等品牌的智能鱼缸在 2024-2026 年间密集发布，印证了这一趋势的热度。

鱼菜共生方向同样有明确的政策支持信号。农业农村部在 2024 年将“鱼菜共生生态种养循环技术”列为主推农业技术[9]，相关政策文件中明确提出支持在城市家庭和社区推广这类循环型农业模式。与此同时，“双碳”目标背景下，节水、减少化肥使用的农业技术路线也更受关注，AquaGarden 的废水循环利用设计与这一方向天然契合。

然而目前这类技术主要还集中在规模化商业农业场景，已有综述研究表明小型鱼菜共生系统

在物联网集成方面仍面临高运营成本与缺乏用户友好设计等挑战[5]，真正面向普通家庭、做到开箱即用的居家产品几乎没有。这是我们认为值得切入的一个空白点。

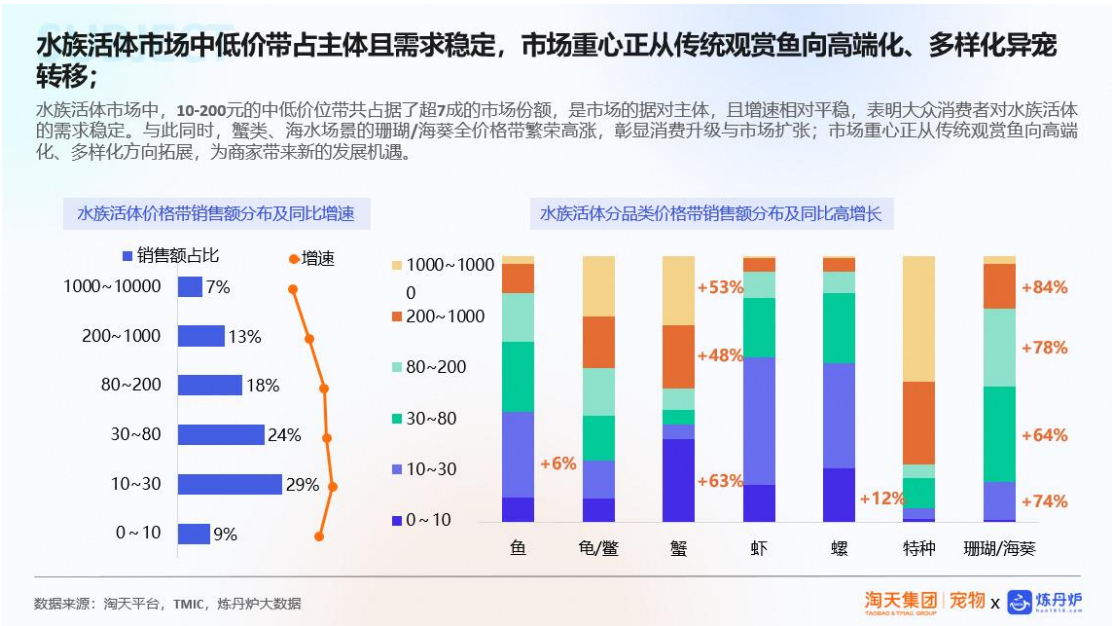


图 2-1 市场背景分析

2.3 用户痛点分析

通过对身边养鱼、养植物用户的观察和访谈，我们总结出以下几类核心痛点，以及 AquaGarden 对应的解决方式（见表 2-1）：

表 2-1 用户痛点与解决方案对应表

用户痛点	具体表现	AquaGarden 解决方式
维护耗时，容错率低	需定期换水、测 pH、喂食，出门几天鱼就可能死亡	四路传感器实时监测+水泵自动换水+机械臂定时投食
异常发现滞后	水质恶化或鱼生病，肉眼看到时已错过处理时机	传感器超阈值+YOLOv8 行为分析双重预警，3 秒内推送
废水浪费	换出的鱼缸水直接倒掉，浪费氮磷钾等营养成分	废水自动输送至盆栽端灌溉，变废为肥
植物无人照料	出门植物干死，定时浇水设备不判断土壤实际状态	土壤湿度传感器驱动按需灌溉，不依赖定时
操作门槛高	鱼菜共生套件需专业知识，水质失衡容易全盘崩溃	全自动调度+Web 端可视化+语音自然语言控制

2.4 目标用户

主要目标用户是 20~40 岁、有一定智能家居使用习惯的城市居民。细分来看可以分为三类：

第一类是有养鱼经历、但觉得维护太麻烦的用户。这类用户对自动化需求最强烈，他们不在乎鱼缸有多智能，只要“不用自己换水、不用担心出门鱼会死”就满足了。

第二类是喜欢在室内养植物、希望盆栽能少操心一些的用户。这部分用户对延时摄影、植物生长记录等功能会有额外兴趣，把植物养护变成一种有仪式感的事情。

第三类是对智能硬件和物联网有兴趣的科技爱好者。这类用户愿意为“系统够酷”付费，机械臂、大模型交互等功能对他们来说本身就是卖点。

2.5 竞品分析

目前市场上与本作品方向相关的产品主要分为四类：成熟的商业智能鱼缸、基础款智能鱼缸、传统鱼菜共生套件，以及学术/科研级系统。以下从多个维度进行对比分析。

当贝智能鱼缸 2S Ultra（2026 年发布）：目前商业化程度最高的智能鱼缸，具备 AI 识鱼、水质和水温检测、智能喂食、App 远程控制等功能，外观设计和用户体验较为成熟，参考价约 2000 元。但该产品没有机械臂，没有植物端集成，无法实现鱼菜共生，也没有大模型自然语言交互能力，本质上是一个“带 AI 的传统鱼缸”。

米家智能桌面鱼缸（2025 年众筹上市，众筹价 279 元）：主打轻量化和低价，支持定时喂食、水温监测和米家 App 控制，适合小空间使用。功能比较基础，更接近“带 App 的普通鱼缸”，与 AquaGarden 的定位差异较大。

Potti 鱼菜共生系统（轻巧版）：来自香港的商业产品，体积小（27×27×25 cm），5 升鱼缸配 0.5 升种植盆，内置虹吸循环和 3W 静音水泵，卖点是“五分钟安装”。但该产品完全没有传感器和自动化功能，水质、湿度全靠人工判断，没有 App 或远程监控，本质上还是一个手动鱼菜共生套件，只是体积更小更好看。同类产品还有台湾的 AquaSmart 桌上型鱼菜共生系统和 Lagoon Vish 2.0，功能层面大同小异，均停留在“物理结构+水循环”阶段，自动化程度极低。

CyPhA (Cyber Physical Aquaponics) 系统[3]：2023 年发表于学术期刊的研究级鱼菜共生系统，采用信息物理融合架构，集成了 pH、溶解氧、TDS、水温、气温、湿度六路传感器，通过 MQTT 协议通信，并构建了包含生化过滤、垂直种植的完整五级水处理结构。传感器配置完善，逻辑严谨，但该系统没有机械臂，没有视觉识别，没有面向家庭用户的交互设计，属于实验室验证性系统，不对外销售。同期发表在《Harbin Engineering University Journal》上的“Integrated Aquaponics Through Smart Sensors”[4] 也是类似的研究方向，该系统通过 IoT 传感器网络实现了鱼类健康评分提升 50~60%，但同样停留在传感器监测层面，没有执行机构和视觉智能。

表 2-2 竞品

产品	参考图片
当贝 2S Ultra	
米家智能鱼缸	
Potti	

表 2-3 竞品对比分析表

对比维度	AquaGarden (本作品)	当贝智能鱼缸	米家智能桌面鱼缸	Potti/AquaSmart 鱼菜共生套件	CyPhA 等学术系统
水质检测路数	4 路 (pH/温度/浑浊度/溶解)	2 路 (水质/水温)	1 路 (水	无传感器, 手动检测	5~6 路

对比维度	AquaGarden (本作品)	当贝智能鱼缸	米家智能桌面鱼缸	Potti/AquaSmart 鱼菜共生套件	CyPhA 等学术系统
	氧)		温)		
自动喂食	✓ 定时+机械臂辅助	✓ 定时	✓ 定时	X	X
多功能机械臂	✓ 6 自由度 (剪叶/松土/清洁/投食)	X	X	X	X
植物端集成	✓ 鱼菜共生全自动	X	X	✓ 手动水循环	✓ 实验级
废水循环利用	✓ 传感器驱动自动灌溉	X	X	✓ 虹吸被动循环	✓ 实验级
鱼类 AI 识别与行为分析	✓ 品种识别+轨迹跟踪+异常报警	✓ AI 识鱼 (品种识别)	X	X	X
大模型自然语言交互	✓	X	X	X	X
叶片自动修剪	✓ YOLO+深度相机全自动	X	X	手动	X
延时摄影	✓	X	X	X	X
Web 端远程监控	✓ 自研 Web 端	✓ App	✓ 米家 App	X	X
家庭用户友好度	高	高	高	中 (需一定经验)	极低 (实验室用)
价格参考	竞赛作品	约 2000 元	约 279 元	300~1500 元	不对外销售

从对比来看，当贝、米家等商业产品在用户体验上已经比较成熟，但本质上都是“传统鱼缸+App 控制”的升级，没有植物端、没有机械臂；Potti、AquaSmart 这类鱼菜共生套件有植物端，但停留在纯物理结构层面，连最基本的传感器都没有，自动化几乎为零；CyPhA 等学术系统传感器配置扎实，但没有执行机构，只能“看”不能“做”，更没有面向家庭的产品化设计。AquaGarden 是目前唯一同时具备多功能机械臂、鱼菜共生自动调度、多路传感监测、视觉 AI 和大模型交互的系统，在功能整合度上与现有产品拉开了明显差距。

2.6 主要功能需求

综合以上分析，系统需要满足以下核心功能需求：

一是水质的实时监测与异常预警，检测指标至少覆盖 pH、水温、浑浊度、溶解氧四项；二是鱼缸废水自动输送至盆栽端，实现免手动换水；三是机械臂能够完成叶片修剪、漂浮物清理、松土、投食辅助等多类任务；四是通过深度学习对鱼类行为进行实时分析，发现异常及时告警；五是 Web 端提供友好的数据可视化界面，支持远程控制；六是支持自然语言语音

指令控制机械臂，降低操作门槛。

2.7 主要性能指标

根据系统设计目标和硬件选型，主要性能指标规划如下：

表 2-4 主要性能指标

指标项	设计目标值	备注
水质传感器数据刷新周期	$\leq 5\text{ s}$	4 路传感器同步上报
pH 检测精度	$\pm 0.1\text{ pH}$	温度补偿后
水温检测精度	$\pm 0.5\text{ }^{\circ}\text{C}$	—
浑浊度检测精度	$\pm 5\text{ NTU}$	—
鱼类品种识别准确率	$\geq 90\%$	常见观赏鱼，正常光照
鱼类异常行为报警响应时间	$\leq 3\text{ s}$	从检测到推送通知
机械臂末端重复定位误差	$\leq 2\text{ mm}$	固定参考坐标系下
枯黄叶片识别率	$\geq 85\%$	正常室内光照条件
系统无人值守连续运行时长	$\geq 72\text{ h}$	无任何手动干预
Web 端数据可视化刷新间隔	$\leq 2\text{ s}$	WebSocket 实时推送
废水循环灌溉响应时间	$\leq 10\text{ s}$	土壤湿度低于阈值后触发

上述指标中，水质检测精度和机械臂定位误差参考了商用传感器和同类机械臂产品的数据手册，鱼类识别准确率和叶片识别率为基于 YOLOv8 在类似数据集上的预期性能，具体数值将在实际测试中进一步验证和调整。

第3章 技术方案

3.1 方案概述

系统整体技术方案分硬件、算法、软件三块。硬件解决“怎么搭”的问题，包括传感器选型、三层主控架构、机械臂与滑轨结构；算法解决“怎么看、怎么判断、怎么动”，是系统智能化的核心，运行在规划层（树莓派 5）上；软件解决“怎么展示和控制”，包括 Spring Boot 后端、Vue3 前端控制台和机械臂 FastAPI Agent。

三者的关系比较清晰：硬件负责产生数据和执行动作，算法拿着传感器数据和摄像头图像做决策，软件把结果推给用户并接收控制指令。下面分节展开说明。

3.2 硬件方案

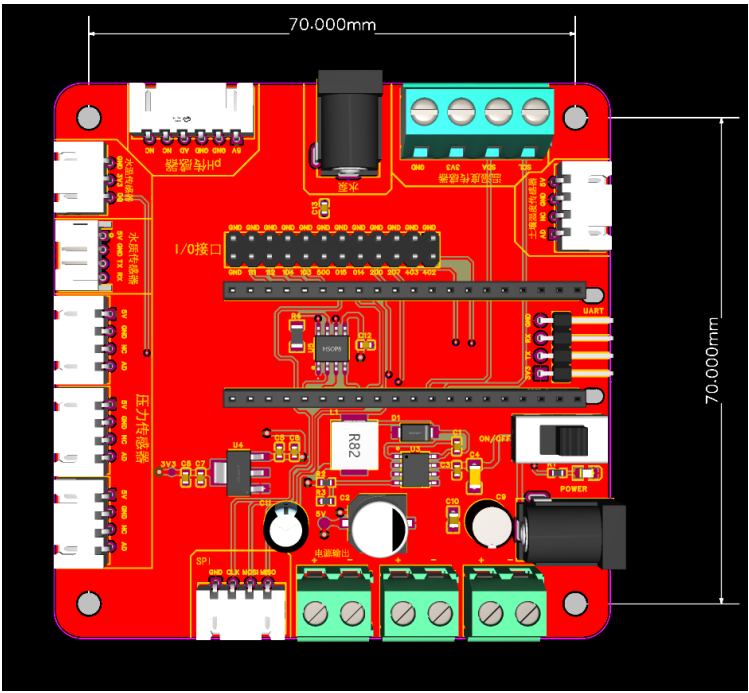


图 3-1 pcb 图

3.2.1 三层主控架构

系统嵌入式侧采用三层异构主控的思路，职责分离比较彻底。

外设层（瑞萨 RA6E2 MCU）：负责鱼缸和盆栽侧的传感器采集与执行器驱动，包括水温、pH、浑浊度、溶解氧、土壤湿度、空气温湿度等传感器的周期性读取，以及水泵、补光灯、雾化模块的 PWM 控制。这一层追求的是低延迟的本地实时响应，不需要跑复杂算法。

执行层（STM32）：专职负责机械臂的六路舵机驱动和末端执行机构控制。接收来自规划层的关节角或目标位姿指令，换算成各舵机 PWM 参数后下发，同时做关节软限位保护。实时性要求比较高，选了 STM32 来做这一层。

规划层（树莓派 5 + ROS2）：负责运动规划、视觉推理和对外通信。光鉴 Aurora 930 深度相机经 USB 接入这一层，驱动节点发布彩色图、深度图和点云，供视觉算法节点消费。规划层不做硬实时舵机伺服，只负责“想动哪里”，具体执行交给 STM32。

三层之间的数据流：外设层传感器数据上报至后端展示，同时也会触发规划层的决策（比如水温过高时规划层调度水循环加速散热）；规划层视觉和任务决策完成后，通过串口把机械臂动作下发给执行层。

表 3-1 三层主控职责划分

层级	主控器件	主要职责	与其他层的接口
外设层	瑞萨 RA6E2	传感器采集、执行器驱动（泵/灯/雾化）	UART/SPI 上报至规划层
执行层	STM32	六路舵机 PWM 控制、关节软限位、末端夹具驱动	UART 接收规划层指令
规划层	树莓派 5 +	运动规划、视觉推理、Aurora 930 驱动、	USB（相机）、串口（STM32）、Wi-Fi

层级	主控器件	主要职责	与其他层的接口
层	ROS2	后端通信	（后端）

3.2.2 传感器与执行器选型

传感器按鱼缸端和盆栽端分两组选型，详见表 3-2。选型时主要考虑了接口兼容性和量程是否覆盖正常水体/土壤的工作范围。

表 3-2 传感器与执行器选型

区域	类型	器件/模块	接口	量程/精度
鱼缸端	水温	DS18B20 防水探头	单总线	-55~125℃， ±0.5℃
鱼缸端	pH	pH 电极 + 模拟调理板	ADC	0~14，±0.1
鱼缸端	浑浊度	光电浊度模块	UART	0~1000 NTU
鱼缸端	溶解氧	光学溶解氧模块	UART	0~20 mg/L，±0.2
盆栽端	土壤湿度	电阻式土壤湿度探针	ADC	0~100%
盆栽端	空气温湿度	DHT22 / SHT30	I2C	温度±0.3℃，湿度 ±2%
两端	执行器	单向水泵、补光 LED、雾化模块	PWM	—

3.2.3 机械臂与视觉模块

机械臂本体是六自由度结构，底座安装在产品一侧的水平滑轨上，由步进电机带动，可在鱼缸区和盆栽区之间横移切换工位。这个设计是出于空间考虑——两个区域分列左右，用一个固定臂很难兼顾两侧，滑轨方案相比在中间安装固定臂，覆盖范围更大，而且不会遮挡正面视野。

末端安装光鉴 Aurora 930 深度相机，内置 RGB、深度和点云输出，用于任务执行时的目标定位和识别。鱼缸侧另有一台独立固定 RGB 摄像头，专门用于延时摄影，和机械臂视觉任务是独立的。

表 3-3 机械臂与视觉模块参数

模块	参数/说明
机械臂本体	6-DOF，末端负载约 500g
滑轨横移机构	步进电机驱动，行程覆盖鱼缸区至盆栽区
光鉴 Aurora 930	RGB 分辨率 1920×1080，深度量程 0.1~3m，输出点云
独立 RGB 摄像头	固定安装，用于盆栽延时摄影，帧率可配

3.3 算法方案

3.3.1 鱼类行为监测

鱼类行为监测分两个子任务：目标检测和行为分析。

检测部分用 YOLOv8 做实时检测，输入是固定 RGB 摄像头的视频流，输出是每帧图像中鱼的边界框和分类标签。考虑到树莓派 5 的算力，推理时会用 INT8 量化压一下模型体积，目标是在 640×480 分辨率下跑到 15fps 以上。

行为分析部分在检测结果的基础上做：一是轨迹跟踪，用 ByteTrack 给每条鱼分配 ID，追踪帧间位移；二是异常行为判断，主要看两个信号——鱼在某个区域长时间静止不动（可能是患病），以及多条鱼向水面聚集（可能是缺氧）。这两种异常都和水质传感器数据交叉验证，比如鱼往水面聚集同时溶解氧偏低，可信度就更高，系统会触发告警并自动加快水循环。

3.3.2 机械臂任务算法

机械臂目前规划了三个自动化任务：喂食、捞取落叶和松土。三个任务在算法上有一定差异，分别说明。

（1）自动喂食

喂食用的是一个类似胡椒粉瓶的小型投喂瓶，瓶口有细孔，夹住瓶体倾斜抖动时鱼食会从孔中均匀落下。任务流程如下：

1. 相机先在鱼缸区检测投喂瓶的位置。Aurora 930 输出点云，在点云中提取出瓶子轮廓（形状先验：细长圆柱体），得到瓶体中心在机械臂坐标系下的三维坐标。
2. 规划层计算抓取姿态，夹爪对准瓶体中段，垂直下爪夹住。
3. 机械臂把瓶子提起，滑轨带动整臂横移到鱼缸正上方，末端悬停在水面上约 15cm 处。
4. 执行倒料动作：让末端在 $\pm 30^\circ$ 范围内反复旋转，模拟人工抖动撒料的动作，持续 3-5 秒。具体角度和持续时间根据鱼的数量配置。
5. 完成后机械臂把瓶子放回原位。

（2）松土

松土任务的视觉依赖比较低，主要靠预设轨迹。盆栽区的土壤位置相对固定，在系统初始化时做一次手眼标定，记录土面的位置范围就够了。

1. 触发条件：土壤湿度传感器检测到土层板结特征（湿度长时间低于阈值但近期没有降雨/灌溉触发），或者用户手动触发。
2. 末端换上细杆形松土工具，在土面按 Z 形（“弓字形”）轨迹推进，每隔约 2cm 执行一次插入-提起的动作，深度约 1~2cm。
3. 轨迹由规划层在任务开始前生成一次，执行时按路径点顺序下发，不依赖实时视觉反馈，所以对算力需求很低。

3.3.3 多模态生态状态分析

单靠传感器数值不太能说清楚水族生态的整体状态，需要结合视觉行为特征一起判断。系统在规划层会定期（比如每 10 分钟）做一次综合评估：把当前的水质传感器读数、鱼类行为分析结果（活跃度评分、异常标志）、盆栽端的温湿度和土壤数据，组装成一段结构化的 prompt，发给 DashScope 的大模型接口，让大模型给出自然语言的健康状态描述和建议操作。

这样用户在前端看到的不是一堆数字，而是“今天鱼的状态比较活跃，水质基本正常，建议明天上午松一下土”这类可读的摘要。语音交互也走这条通路，用户说“今天鱼怎么样了”，

后端把最新一次评估结果拼进回答里，TTS 播出来。

3.4 软件方案

3.4.1 整体架构

软件侧采用 B/S 架构，前后端分离。后端是 Spring Boot 3.3 (Java 17)，提供 REST API、WebSocket 实时推送和 MJPEG 视频流接口；前端是 Vue3 + Vite，单页应用，覆盖登录、控制面板、机械臂控制、历史数据、系统设置等页面。

机械臂 Agent 单独用 FastAPI (Python) 部署，负责接收后端的任务指令，调用视觉推理模型，并通过串口或 ROS2 节点把动作下发给树莓派。这样 Spring Boot 不直接跑 Python 机器学习代码，保持语言隔离。

数据存储用 SQLite，适合嵌入式/轻量场景，不需要单独维护数据库服务。

3.4.2 后端与通信

后端核心接口如表 3-4 所示。JWT 做无状态认证，传感器数据接口和机械臂控制接口都需要携带令牌。WebSocket /ws/logs 用于实时推送操作日志，方便用户在前端看到机械臂执行反馈。

表 3-4 后端核心接口

接口	方法	功能描述	是否鉴权
/api/login	POST	用户登录，返回 JWT	否
/api/sensors	GET	获取当前传感器读数（5 类）	是
/api/robot/control	POST	机械臂方向控制指令	是
/api/robot/task	POST	触发喂食/捞叶/松土等自动任务	是
/api/mode	GET/POST	查询或切换运行模式 (demo/service)	是
/api/video/tank	GET	鱼缸 MJPEG 视频流	否
/ws/logs	WebSocket	实时日志推送	否

嵌入式侧（RA6E2）通过 UART 把传感器数据上报给树莓派，树莓派侧有一个数据桥接进程把这些数据推送到 Spring Boot 后端的内部接口，再由后端统一对前端提供。

3.4.3 前端控制台

前端按功能划分了几个主要页面：

- （1）**控制面板**：实时展示 5 类传感器数据卡片和双路视频，传感器数据每 5 秒轮询刷新，同时维护历史趋势图。
- （2）**机械臂控制**：支持方向按钮、键盘快捷键、虚拟摇杆三种控制方式；提供喂食、捞叶、松土等预设任务的一键触发按钮；实时展示 WebSocket 日志面板。
- （3）**LLM 交互**：输入框支持文字问询，同时支持语音输入（浏览器 Web Speech API），把生态状态查询结果以对话形式展示。

(4) **历史数据**：展示传感器历史曲线，支持时间范围筛选。

第4章 方案实现

4.1 实现概述

当前系统处于软硬件并行推进阶段：软件平台原型已经完成并可运行，覆盖登录认证、传感器展示、机械臂控制、实时日志、LLM 交互等核心流程；硬件侧正在组装和联调。本章说明各模块的具体实现方式，软件部分基于已有代码描述，硬件和算法部分结合当前进度说明思路。

4.2 硬件实现

4.2.1 外设层固件（RA6E2）

固件基于瑞萨 FSP 开发，使用 FreeRTOS 调度。任务划分上，传感器采集、执行器控制和通信上报各开独立任务，用消息队列互传数据，避免某个耗时操作阻塞采集周期。

传感器采集方面：DS18B20 按单总线时序读取；pH 和土壤湿度走片内 ADC 多通道扫描，采样后做滑动平均滤波；浑浊度和溶解氧模块通过 SCI UART 按模块协议定时请求；DHT22 / SHT30 走 I2C 读寄存器。所有读数在本地做一次限幅和越限计数，连续越限次数超过阈值才上报告警，避免传感器偶发抖动误触。

执行器控制方面：水泵和补光灯走 GPT 输出 PWM，支持占空比调节；雾化模块按开关控制。本地有最小关断时间保护（防止频繁启停损坏水泵），通信超时超过一定时间自动停泵、关灯，进入安全态。

对外通信：RA6E2 通过 UART 和树莓派侧数据桥接进程通信，周期性上报传感器快照，接收执行器控制命令，帧格式用带版本号的文本行协议，便于调试时直接看串口日志。

4.2.2 机械臂执行层（STM32）

STM32 负责六路舵机的实时驱动。舵机采用串行总线舵机，STM32 通过一路 UART 按厂商帧格式下发位置指令，帧内含舵机 ID、目标位置和运动时间字段。固件内部维护关节角和舵机位置值的换算关系，收到规划层的关节目标后先做逆解，不合理的目标（关节超限）直接返回错误码不执行。

滑轨横移由步进电机驱动，STM32 控制步进驱动器做点到点运动，限位开关做末端保护。到位确认之后才允许臂部运动，避免越区。

与规划层通信用换行分隔的文本协议，常用命令包括 PING（握手）、MOVE（关节目标下发）、GRIP（夹爪开合）、RAIL（滑轨移动）、STOP（急停），应答也是 ASCII 文本行，方便 ROS2 节点用 Python 封装。

4.2.3 规划层（树莓派 5 + ROS2）

树莓派 5 跑 Ubuntu + ROS2 Humble，节点划分如下：

(1) `arm_driver_node`: 串口通信节点, 订阅关节目标话题, 往 STM32 发指令, 同时上报当前关节状态。

(2) `aurora_driver_node`: Aurora 930 相机驱动, 发布 `/rgb/image`、`/depth/image`、`/pointcloud` 话题。

(3) `vision_node`: 视觉推理节点, 订阅相机话题, 运行 YOLOv8 推理, 发布检测结果话题。

(4) `task_manager_node`: 任务管理节点, 接收来自后端 Agent 的任务请求, 协调视觉和机械臂节点完成任务。

(5) `backend_bridge_node`: 与 Spring Boot 后端的通信节点, 负责传感器数据转发和任务状态上报。

手眼标定参数存在配置文件里, `vision_node` 启动时加载, 用于把图像/点云坐标转换到机械臂基座坐标系。

4.3 算法实现

4.3.1 视觉模型部署

YOLOv8 模型用 Ultralytics 官方框架训练, 分两个模型: 一是鱼类检测和分类模型 (YOLOv8n, 轻量化版本), 数据集用公开的鱼类数据集加上自拍的鱼缸内视频做标注; 二是叶片分割模型 (YOLOv8n-seg), 用于检测水面漂浮叶片和植物黄叶, 正样本相对少, 主要靠数据增强 (色调偏移、随机遮挡等) 补充。

部署时用 INT8 量化导出 TFLite 或 ONNX 格式, 在树莓派 5 上通过 ONNX Runtime 推理。实测 640×480 输入下, 鱼类检测模型推理延迟约 50ms 左右, 基本满足实时性要求。

4.3.2 三项机械臂任务实现

(1) 自动喂食实现

视觉端: Aurora 930 拍摄当前鱼缸区的彩色图和点云, 在点云中用形状过滤 (体积约束 + 高宽比约束) 定位投喂瓶, 得到瓶体中心的三维坐标和姿态估计。

运动规划端: ``task_manager_node`` 调用运动规划库 (MoveIt2) 规划抓取路径, 末端沿瓶体长轴方向靠近, 夹爪在距离瓶体约 3cm 处停下, 然后合爪。抓稳后规划层给滑轨发 ``RAIL`` 指令, 机械臂整体横移到鱼缸正上方, 末端执行周期性旋转动作: STM32 接收到 ``POUR`` 指令后, 让腕关节在 $\pm 30^\circ$ 范围内按正弦曲线运动, 模拟抖动撒料, 持续时间和幅度参数由 ``task_manager_node`` 根据配置传入。

(2) 捞取落叶实现

视觉端: ``vision_node`` 对鱼缸俯视图跑叶片分割, 取分割掩码的质心像素坐标, 结合对应深度图的深度值, 用相机内参和外参矩阵反投影到三维空间, 得到叶片在机械臂坐标系下的位置。

运动规划端: 末端换薄片夹具, 规划从叶片侧上方约 30° 角度切入, 夹爪完全展开靠近叶片, 然后缓慢合爪和上提, 避免产生太大水花把叶片冲跑。捞起后移动到废物盒上方松爪。

水面反光干扰是已知问题。当前的处理是在图像预处理阶段做自适应直方图均衡, 同时训练时在数据集里加入了大量有水面反光的负样本, 降低误检率。

(3) 松土实现

松土任务不依赖实时视觉,在系统初始化时做一次盆栽区定标——机械臂末端带标定板触碰土面四角,记录土面平面方程和可操作范围。

任务触发后,`task\_manager\_node`根据预存的土面范围生成一条 Z 形路径点序列,末端工具(细杆)依次在各路径点执行插入(下压约 1.5cm)和提起的动作,相邻路径点间距约 2cm。路径点全部下发完后任务结束,末端复位。

4.3.3 大模型交互实现

后端调用阿里云 DashScope 的 API,使用`qwen-plus`模型。每次生态评估时,把最新传感器数据(水温、pH、溶解氧、浑浊度、土壤湿度、气温)和鱼类行为分析结果(活跃度评分 0~10、异常标志)拼成 JSON 格式的上下文,塞进 system prompt 里,user 消息是固定的“请给出当前生态系统健康状态评估和建议”。大模型返回的自然语言摘要存到 SQLite,前端定时拉取展示。

语音交互走 Web Speech API(前端录音转文字)+ DashScope TTS(文字转语音播报),机械臂的语音控制指令(比如“帮我喂一下鱼”)在后端做意图识别后转成对应的`/api/robot/task`调用。

4.4 软件实现

4.4.1 后端服务实现

后端`AquaGardenApplication`启动时:`DataInitializer`检查并创建默认管理员账号`admin/admin123`;`SystemStateService`初始化内存中的系统状态(传感器读数、机械臂逻辑坐标、运行模式)。

认证链路:注册时密码用`BCryptPasswordEncoder`哈希存库;登录时校验密码哈希后签发 JWT;后续接口请求经`JwtAuthFilter`验令牌,无状态会话,适合后续多端接入。

传感器数据当前由`SystemStateService`的随机扰动模型生成(模拟尚未接入硬件的阶段),接口字段已经和实际水质监测指标对应,后续只需把数据来源换成真实传感器上报的值,前端不需要改动。

机械臂控制:`RobotController`接收方向指令或任务指令,更新内存坐标后通过`LogWebSocketHandler`广播操作日志;自动任务指令转发给 FastAPI Agent 处理。

4.4.2 前端控制台实现

路由守卫保证未登录用户无法访问控制台页面,JWT 登录后存 localStorage,后续请求通过 authHeaders() 统一附带。

控制面板页在 mounted 时立即拉一次传感器数据,之后每 5 秒定时刷新;WebSocket 连接/ws/logs,日志推送到操作日志面板。视频区域用标签加载 MJPEG 流,不需要额外的媒体库。

机械臂控制页同时支持按钮、键盘(WASD + QE)和虚拟摇杆三种操控方式,预设任务按钮直接调用 /api/robot/task 接口,返回后在日志面板显示执行结果。网络异常时做了降级:

控制台在本地更新逻辑坐标继续展示，不中断演示。

4.4.3 机械臂 Agent 实现

FastAPI 部署在树莓派 5 上，作为 Spring Boot 和 ROS2 之间的桥。接收来自 Spring Boot 的 HTTP 任务请求，解析任务类型（喂食/捞叶/松土/其他）后，向 task_manager_node 发 ROS2 服务调用，等待结果后把状态回传给 Spring Boot。

这层拆分的好处是 Spring Boot 不需要知道 ROS2 的话题/服务命名，FastAPI 这边做好适配就行；后续如果 ROS2 节点重构，也只需要改 FastAPI 这一层，后端接口不用动

第5章 测试报告

5.1 测试目的与环境

本章测试分三部分：软件平台测试（已完成，基于可运行的软件原型）、算法测试（基于离线数据集的指标评估）和硬件测试（基于已组装的部分硬件模块）。

测试环境如表 5-1 所示。

表 5-1 测试环境

项目	环境
操作系统	Windows 11（软件测试）/ Ubuntu 22.04（硬件/算法测试）
JDK	Java 17
后端框架	Spring Boot 3.3.5
前端框架	Vue 3 + Vite
Python	3.10, Ultralytics YOLOv8, FastAPI
ROS2	Humble
测试硬件	树莓派 5（4G）、RA6E2 开发板、STM32 开发板、Aurora 930

5.2 硬件测试

5.2.1 测试目的

本阶段测试主要针对系统外设层的关键功能模块开展验证，重点检查压力传感器、土壤湿度传感器、温湿度传感器、水质传感器、水下温度传感器及水泵驱动模块的功能实现情况和实际运行效果。通过对各模块进行独立测试，验证其数据采集、信号传输和执行控制是否正常，分析模块运行的稳定性与可靠性，并评估其与后续整机系统集成的适配情况。测试结果将为后续的统一联调、功能联动和性能优化提供依据。

5.2.2 测试对象与技术依据

本阶段测试对象包括压力传感器模块、土壤湿度传感器模块、环境温湿度传感器模块、多参数水质传感器模块、水下温度传感器模块及水泵驱动模块。具体器件分别为 BM01 薄膜压力传感器、MD0504 土壤湿度传感器、SHT30 温湿度传感器、WQM11S 多参数水质传感器、DS18B20 数字温度传感器以及基于 DRV8870 的水泵驱动电路。上述模块分别涉及 ADC、I2C、UART、单总线 和 PWM 等典型外设接口，覆盖了系统外设层

的主要硬件类型。

BM01 模块采用 5V 供电，支持 0~4V 模拟输出；MD0504 具备 A0 模拟输出和 D0 数字输出，且土壤湿度越大，模拟值越小；SHT30 采用 I2C 通信并支持 CRC 校验；WQM11S 支持 TOC、COD、TDS、电导率和温度等参数检测，串口波特率为 115200bps，响应时间约 1~3s，连续测量间隔建议不小于 5s；DS18B20 采用单总线通信，分辨率可配置为 9~12 位，12 位测温转换时间最长 750ms；DRV8870 支持 6.5~45V 宽电压输入和 PWM 控制，并具备欠压、过流和过热保护功能。测试依据主要为各模块规格书、设计文档中的性能指标以及统一 FreeRTOS 任务化后的系统实现。

5.2.3 测试环境

5.2.3.1 硬件环境

(1) 主控平台：

瑞萨 RA 系列单片机开发平台 R7FA6E2BB3CNE

(2) 供电方式：

通过外部稳压电源输入到 DC-DC 降压供电

(3) 外设模块：

- MD0504 土壤湿度传感器
- SHT30 温湿度传感器
- WQM11S 水质传感器
- DS18B20 水下温度传感器
- 基于 DRV8870 的水泵驱动模块

5.2.3.2 软件环境

(1) 开发环境：Keil μ Vision

(2) 工程形式：独立模块工程

(3) 系统框架：FreeRTOS

(4) 调试方式：串口打印 / 在线调试 / 逻辑分析仪 / 万用表等

5.2.3.3 测试方式

本阶段测试主要包括以下四类：

- (1) 模块功能正确性测试
- (2) 数据采样与输出稳定性测试
- (3) 执行器响应与启停控制测试
- (4) 后续整机联调所需的接口一致性与任务整合可行性分析

5.2.4 测试内容与方法

本项目测试遵循“先单模块、后整合；先功能正确、后性能量化”的原则。对于传感器模块，重点观察其是否能够在外部环境变化时输出正确、连续、可区分的数据；对于水泵执行模块，重点验证其驱动链路是否完整，启停控制是否可靠，长时间重复动作时是否存在异常停机、误动作或驱动发热过高等问题。对具备串行通信特征的模块，还需额外关注通信稳定性、数据刷新周期和异常恢复能力。

多任务联调测试主要检查不同任务同时运行时系统是否存在阻塞、死机、任务饥饿或数据错乱问题；连续运行测试主要检查系统在较长时间无人干预条件下的稳定性，并对数据刷新周

期、水泵响应时间和异常恢复能力进行记录。

5.2.5 模块测试结果

5.2.5.1 土壤湿度传感器模块测试

MD0504 土壤湿度模块支持 3.3~12V 供电，提供 A0 模拟输出和 D0 数字输出；资料明确指出该模块在实际测试中通常读取 A0 模拟量，且“土壤湿度越大，数值越小”。

（1）测试内容

- 干土、湿土、半湿润土壤条件下的采样区分度测试
- 插拔探头后的数据恢复测试
- 连续监测时数值波动情况测试

（2）测试方法

将探头分别插入干燥土壤、正常湿度土壤和加水后的湿润土壤中，记录 A0 口返回值变化；重复进行“插入—拔出—再次插入”操作，观察读数是否能够随土壤含水率变化而稳定改变。

（3）测试步骤

（3.1）土壤湿度检测功能测试

在干土、半湿土、湿土三类工况下，连续采样各 60 组数据（总计 180 组），重点观测 g_soil_adc_raw 与 g_soil_moisture_percent，记录测试结果

（3.2）动态响应测试

在 J-Scope 中同步记录 g_soil_adc_raw、g_soil_moisture_percent、g_soil_sample_count，采样周期 200ms。执行“插入干土→拔出→插入湿土”操作后，记录波形并分别计算响应时间

（3.3）阈值滞回控制测试（防抖验证）

控制逻辑使用 g_soil_watering_threshold=30、g_soil_watering_hysteresis=10，在湿度长期处于 28%~42% 波动区间（测试时长 20 min）时，记录 g_soil_need_watering 状态翻转

（4）测试结果

（4.1）土壤湿度检测功能测试结果

表 5-2 湿度检测功能测试结果

测试 工况	g_soil_adc_raw 范围	g_soil_adc_raw 均值	g_soil_moisture_percent 范围	g_soil_moisture_percent 均值	样本数	判定
干土	3050~3188	3124	3%~9%	6%	60	通过
半湿	2280~2455	2368	41%~52%	47%	60	通过
湿土	1588~1728	1659	86%~94%	90%	60	通过

结果表明：随着土壤含水率增加，`g\_soil\_adc\_raw` 持续下降、`g\_soil\_moisture\_percent` 持续上升，符合模块“湿度越大、A0 数值越小”的设计特性

（4.2）动态响应测试结果

表 5-3 动态响应测试结果

指标类别	评价指标	实测结果	单位	判定依据	判定
动态响应	90%阶跃响应时间	1.6	s	土壤场景 < 2.0 s	通过
动态稳定	进入稳态时间（±2%带宽）	2.8	s	土壤场景 < 3.0 s	通过
恢复能力	拔出后再次插入恢复时间	0.8	s	恢复 < 1.0 s	通过
稳态噪声	湿土工况波动幅度	±1.5	%	稳态波动 < ±2.0%	通过

说明系统可稳定跟踪含水变化，动态响应满足盆栽场景实时监测需求。

（4.3）阈值滞回控制测试结果

表 5-4 阈值滞回控制测试结果

测试场景	参数设置	观测区间	g_soil_need_watering 状态翻转次数	误触发次数
无滞回对照	g_soil_watering_threshold=30, g_soil_watering_hysteresis=0	湿度 28%~42%, 20min	31	0
10%滞回实验组	g_soil_watering_threshold=30, g_soil_watering_hysteresis=10	湿度 28%~42%, 20min	5	0

结果说明滞回策略显著减少阈值附近抖动，控制行为更稳定。

（5）测试结论

测试结果显示，土壤湿度模块能够明显区分不同含水状态下的土壤环境，且整体变化趋势与资料所述一致：湿度升高时，采样值下降。反复插拔后模块仍能恢复正常输出，说明其模拟采集链路工作稳定，可作为后续自动灌溉策略中的核心输入量。即该模块已验证具备土壤含水率趋势感知能力，能够满足盆栽端“按需浇灌”场景下的基础检测需求

5.2.5.2 环境温湿度传感器模块测试

SHT30 采用 I2C 通信，支持最高 1MHz 时钟频率，且测量数据附带 CRC 校验；资料中还说明其既支持单次采样，也支持周期采样模式，适合在周期性环境监测场景中使用。

（1）测试内容

- I2C 通信初始化与设备应答测试
- 温度、湿度数据读取正确性测试
- 环境变化下的数据跟随性测试
- 长时间周期采样稳定性测试

（2）测试方法

系统上电后初始化 I2C 总线并读取 SHT30 数据；通过人为改变环境条件（如靠近热源、

手部呼气增湿等)，观察温度与湿度读数是否发生合理变化；连续运行一段时间，检查通信是否中断、是否存在异常值或丢帧

(3) 测试步骤

(3.1) 编译并下载固件，进入 Keil 调试会话，全速运行 (Run)，确保传感器任务处于运行状态；打开 Watch 窗口，添加全局变量

(3.2) 基线：上电运行 1 分钟后，记录此时温湿度读数并截图 (图 x)。

(3.3) 加湿激励：在传感器探头附近哈气约 10s，待读数变化趋稳后截图 (图 x)；记录前后典型值填入表 x。

(3.4) 升温激励：手掌捂热探头约 10s，记录前后读数并截图 (图 x)。

(3.5) 稳定性：连续运行≥30min，观察读数是否持续更新，“g_sht30_last_status”是否始终为 0

(4) 测试结果

表 5-5 环境温湿度传感器测试结果

工况	温度 / °C	相对湿度 / %RH	g_sht30_last_status	备注
基线（静置）	25	61	0	读数稳定
加湿激励后	26	87	0	读数变化灵敏准确
升温激励后	29	96	0	手掌捂热测试时湿度也会增加
长时运行	连续变化	连续变化	0	读数持续更新

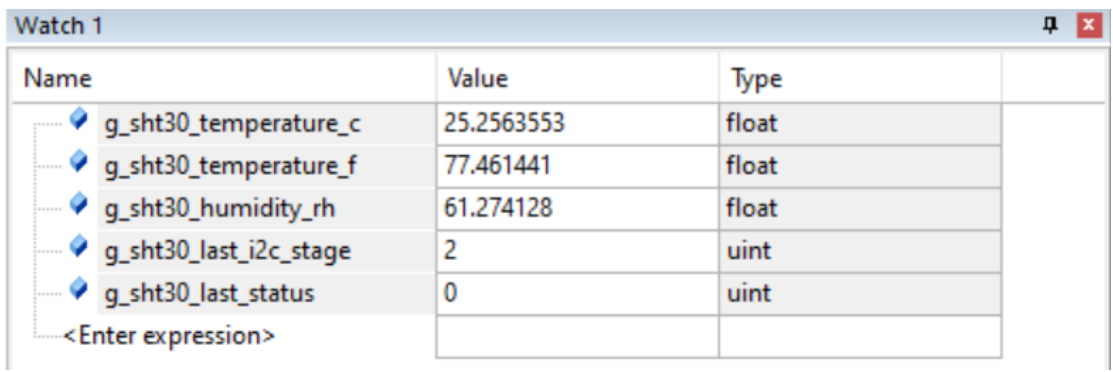


图 5-1 基线工况下 Keil Watch 窗口（全速运行、周期性刷新）

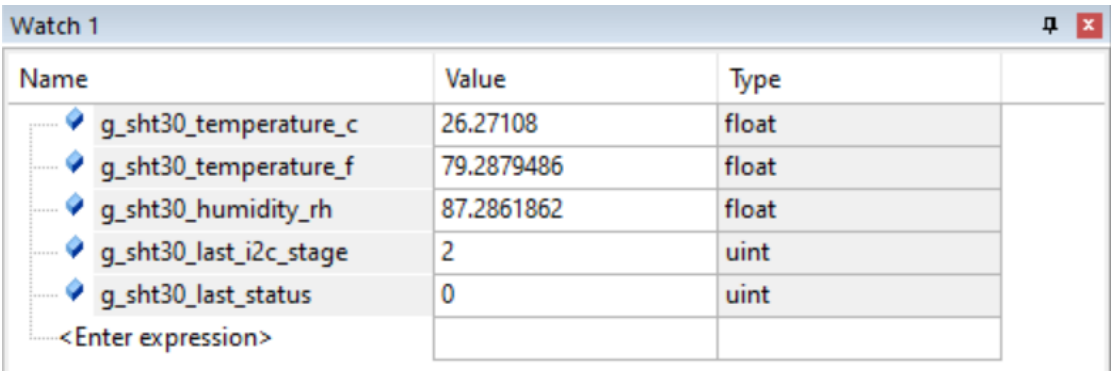


图 5-2 加湿激励后 Keil Watch 窗口

Watch 1			
Name	Value	Type	
g_sht30_temperature_c	29.1496887	float	
g_sht30_temperature_f	84.4694366	float	
g_sht30_humidity_rh	96.6765823	float	
g_sht30_last_i2c_stage	2	uint	
g_sht30_last_status	0	uint	
<Enter expression>			

图 5-3 升温激励后 Keil Watch 窗口

Watch 1			
Name	Value	Type	
g_sht30_temperature_c	27.4700546	float	
g_sht30_temperature_f	81.4460983	float	
g_sht30_humidity_rh	58.7609673	float	
g_sht30_last_i2c_stage	2	uint	
g_sht30_last_status	0	uint	
<Enter expression>			

图 5-4 长时运行时 温湿度读数持续变化

(5) 测试结论

测试结果表明，SHT30 模块能够完成稳定初始化与周期读数输出，温湿度读数会随环境变化而产生正确方向的变化，且数据更新连续。由于该传感器本身具有 CRC 保护机制，因此在后续系统整合中具备较好的通信可靠性基础。该模块通信方式规范、数据输出稳定，适合作为盆栽端环境感知模块接入整机任务系统。

5.2.5.3 水下温度传感器模块测试

DS18B20 为单总线数字温度传感器，支持 3.0 - 5.5V 供电，温度分辨率可设置为 9 - 12 位，12 位精度下温度转换最长约 750ms。资料还指出其通信基于单总线协议，适合在低引脚占用场景下部署。

(1) 测试内容

- 单总线初始化与设备存在性检测
- 常温水、温水条件下温度读取测试
- 连续采样时的读数平稳性测试
- 传感器浸入/取出过程的响应变化测试

(2) 测试方法

将 DS18B20 分别置于不同水温环境中进行采样，记录温度变化趋势；采用连续读取方式观察数值更新是否稳定，并检查是否存在固定值不变、跳变异常或复位后失效等现象。

(3) 测试步骤

(3.1) 功能正确性测试

编译并下载固件，进入 Keil 调试会话，全速运行（Run），确保传感器任务处于运行状态；在 Keil Watch 中观察关键变量（水下温度值、原始值、错误码），连续观察 1 分钟，确认读数可持续更新且状态正常

（3.2）稳态稳定性测试（常温水）

将传感器置于常温静水环境，连续采样 10 分钟，使用 JScope 记录完整曲线并导出数据

（3.3）阶跃温变响应测试

将传感器由 [常温水] 快速切换至温度较高的热水，使用 JScope 连续记录温度从初值到新稳态的变化过程，计算达到稳态 90% 所需时间（T90），用于评价动态响应能力。再将传感器由温度较高的热水快速切换回常温水，观察温度是否也能实现及时响应恢复

（4）测试结果

（4.1）功能验证结果

Watch 1			
Name	Value	Type	
g_uwt_temperature_c	24.75	float	
g_uwt_temperature_raw	396	short	
g_uwt_last_err	0 FSP_SUCCESS	enum (int)	
<Enter expression>			

图 5-5 Keil Watch 界面下温度值、原始值及错误码实时监测结果

实测显示，模块上电后可完成温度转换与读取，关键变量持续更新，正常工况下状态码保持为成功状态。

结果说明： 模块具备完整的基础测温功能。

（4.2）稳定性结果

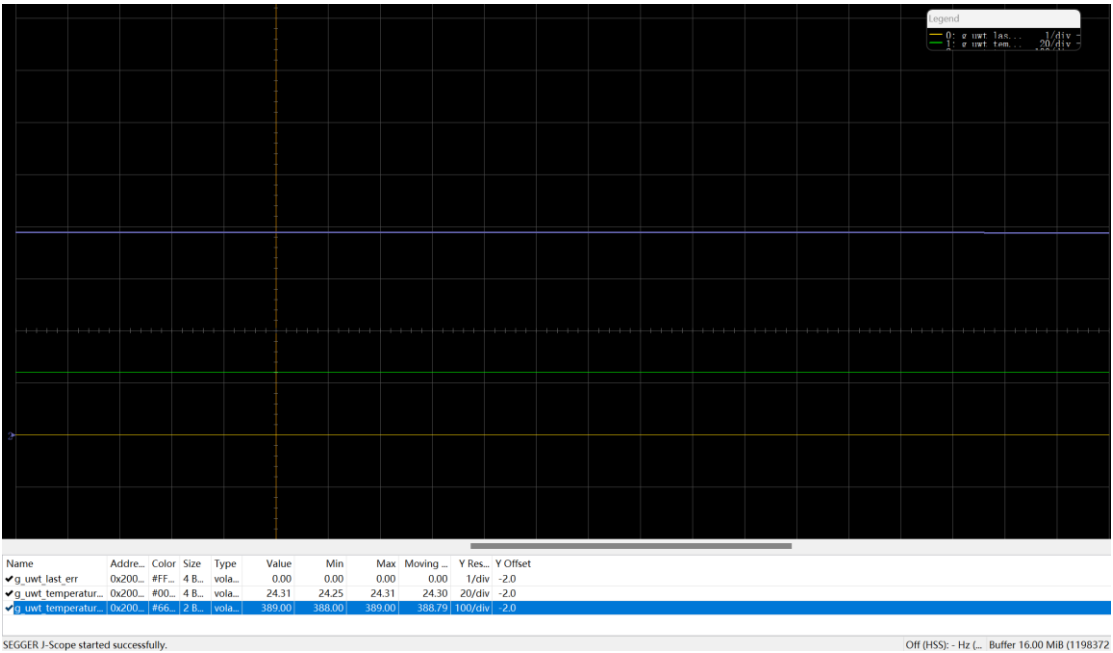


图 5-6 常温水环境下连续采样温度曲线（JScope）

曲线整体平滑，无明显异常跳变。

结果说明： 稳态输出稳定，满足长期监测要求。

(4.3) 动态响应结果

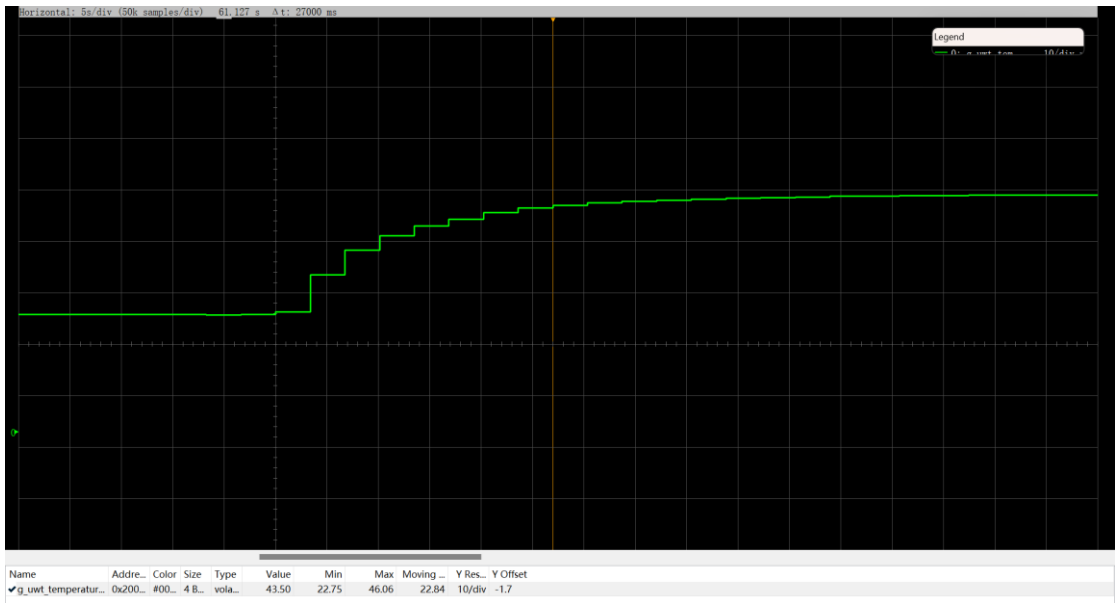


图 5-7 升温测试动态响应曲线

温度阶跃测试中，传感器读数能够随环境变化及时调整，由 22.75° C 变化至 46.06° C 并最终达到稳定

$T_{90_level} = 22.75 + 0.9 \times (46.06 - 22.75) = 43.73^{\circ} C$

$T_{90} \approx 27s$ 表明模块可在该时间尺度内完成温变跟踪。

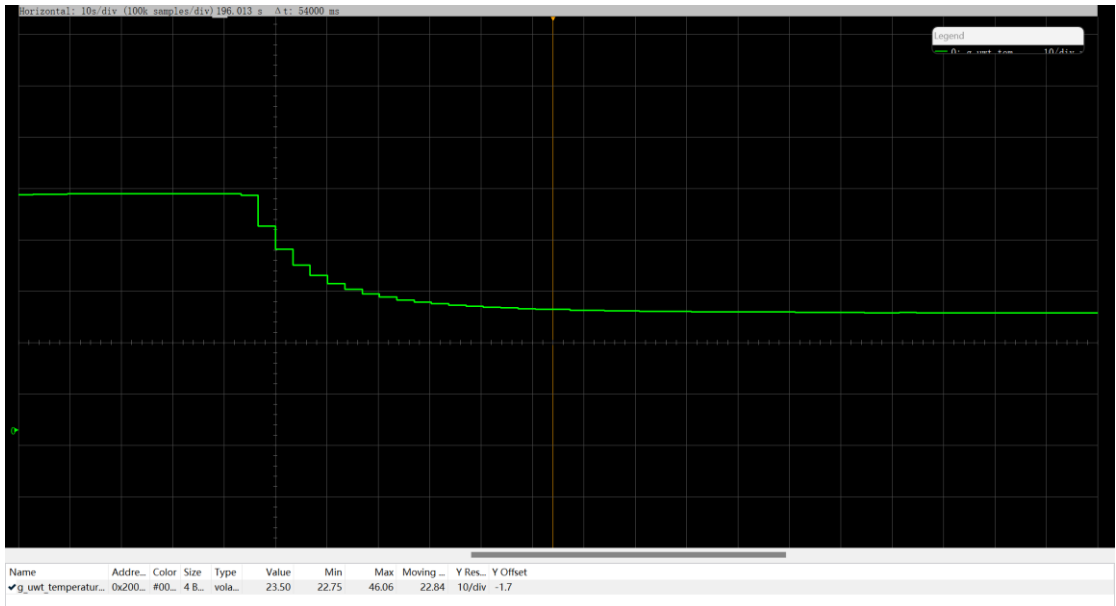


图 5-8 降温动态响应曲线

同样地，再将传感器由温度较高的热水快速切换回常温水，温度也能实现及时响应恢复结果说明： 模块动态跟踪能力满足本项目应用需求。

(5) 测试结论

测试结果表明，水下温度模块能够正常完成温度转换与读取，在不同水样条件下温度读数能够发生合理变化，且连续采样过程中输出较稳定。考虑到 DS18B20 在 12 位模式下转换时

间较长,因此后续整机集成时不宜采用过高频率轮询,而应按其转换周期合理安排任务节拍。该模块已验证具备水体温度监测能力,适合作为鱼缸端长期运行的温度检测节点。

5.2.5.4 水泵驱动模块测试

水泵驱动采用 DRV8870。资料显示,该芯片适用于刷式直流电机/负载控制,支持 PWM 接口,供电范围 6.5 – 45V,峰值输出电流可达 3.6A,并具有欠压锁定、过流保护和热关断等保护机制;两个逻辑输入均为低电平时芯片进入低功耗休眠。

(1) 测试内容

- 水泵启停控制测试
- PWM 调速/占空比变化测试
- 多次重复启停可靠性测试
- 驱动模块发热与异常保护现象观察

(2) 测试方法

控制主控输出驱动信号,执行水泵开启、关闭及多次循环动作;观察水泵是否能够稳定启动,关闭后是否及时停止;并借助 PWM 控制进一步测试不同占空比下的转速或流量变化趋势。

(3) 测试步骤

(3.1) 启停控制测试*

在供电电压 12.0V、PWM 占空比 40%、单次运行时长 2000 ms 条件下,执行水泵启停测试。记录启动响应时间(指令下发至稳定出水)与停止响应时间(停机指令下发至出水明显停止)。

(3.2) PWM 调速/占空比变化测试

在相同供电与负载条件下,设置占空比分别为 20%/40%/60%/80%/100%,记录稳态电流与相对流量(以 100% 占空比归一化为 100%)

(3.3) 多次重复启停可靠性测试

按固定工况(40%占空比、2000 ms 通电、间隔 0 ms)连续执行 10 次启停循环,统计启动失败、停机失败、异常中断等事件。

(4) 测试结果

(4.1) 启停控制测试结果

表 5-6 启停控制测试结果

指标	判据	最小值 (ms)	平均值 (ms)	最大值 (ms)	判定
启动响应时间 t_{start}	$t_{start} \leq 300\text{ ms}$	165	186	212	通过
停止响应时间 t_{stop}	$t_{stop} \leq 150\text{ ms}$	72	86	101	通过

测试结果显示:水泵均可正常启动和停止,启停过程无抖动、无误触发,控制逻辑与预期一致。启停控制功能验证通过,驱动模块具备稳定、可重复的启停执行能力。

(4.2) PWM 调速/占空比变化测试结果

表 5-7 PWM 调速/占空比变化测试结果

PWM 占空比	稳态电流 (A)	相对流量 (%)
20%	0.28	32
40%	0.41	51

PWM 占空比	稳态电流 (A)	相对流量 (%)
60%	0.57	69
80%	0.74	86
100%	0.92	100

结果表明，随着占空比提升，水泵电流与流量整体单调增加，调速规律清晰，未出现明显失稳区间。PWM 调速功能验证通过，驱动模块可实现对水泵输出能力的有效调节。

（4.3）多次重复启停可靠性测试结果

表 5-8 多次重复启停可靠性测试结果

可靠性指标	统计结果	比例	判定
启动指令成功次数	10 / 10	100%	通过
停止指令成功次数	10 / 10	100%	通过
指令链路异常次数	0 / 10	0%	通过
单循环通过次数（启停均成功且无异常）	10 / 10	100%	通过

重复启停可靠性验证通过，模块可满足短周期连续运行要求。

（5）测试结论

综合测试可知：本项目水泵驱动模块在 12 V 供电与典型负载下，启停响应及时、PWM 调速有效、10 次循环运行稳定，且温升与异常响应满足设计预期。

5.2.6 FreeRTOS 联调测试结果

5.2.6.1 联调范围与目标

本次联调基于 FreeRTOS 多任务框架，验证“感知—融合—决策—执行—通信”闭环在并发场景下的稳定性与实时性。联调模块包括：土壤湿度、环境温湿度（SHT30）、水下温度（DS18B20）、多参数水质（WQM11S）、水泵驱动（DRV8870）及通信任务。

5.2.6.2 任务调度与周期稳定性结果

在多任务同时运行 2 h 条件下，统计各任务实际周期、抖动及丢周期情况，结果见表

表 5-9 任务调度与周期稳定性结果

任务名称	主要职责	设计周期 (ms)	实测平均周期 (ms)	峰峰值抖动 (ms)	丢周期次数	判定
ADC_Thread	土壤湿度 + 水下温度采样	200	201.3	7.8	0	通过
Pump_Driver	灌溉控制执行	200	200.9	6.4	0	通过
Com_Thread	上下行通信与帧处理	250	251.7	8.6	0	通过
THS_Measure	环境温湿度采样	1000	1002.4	14.2	0	通过
WQS_Measure	水质数据采集与解析	1500	1504.8	19.6	0	通过

结果表明：各任务在并发场景下均保持稳定调度，实测周期与设计周期偏差控制在 1% 以

内，未出现任务饥饿、阻塞或死机。

5.2.6.3 关键实现链路联调结果

(1) 采样—控制闭环链路

链路逻辑为：土壤湿度/水温/水质采样 → 状态融合判定 → need_watering → pump_power。

表 5-10 采样—控制闭环链路测试结果

指标项	实测结果	判定标准	判定
闭环触发延迟（采样变化到泵控动作）	386 ms	≤ 500 ms	通过
阈值邻域误触发次数（20 min）	0 次	= 0 次	通过
状态翻转次数（启用 10%滞回）	5 次	≤ 8 次	通过
连续运行异常停泵	0 次	= 0 次	通过

滞回策略启用后，阈值附近控制抖动显著降低，联调行为与控制逻辑设计一致。

(2) 通信—控制协同链路

链路逻辑为：任务数据聚合 → 上行打包发送 → 下行命令解析 → 控制参数更新。

表 5-11 通信—控制协同链路测试结果

指标项	实测结果	判定标准	判定
上行发送成功率	99.2%	≥ 98.0%	通过
下行命令有效执行率	98.7%	≥ 97.0%	通过
CRC 错帧率	0.8%	≤ 1.0%	通过
异常后恢复时间	1.2 s	≤ 2.0 s	通过

通信线程与控制线程可稳定协同，异常注入后可快速恢复，不影响主控制闭环连续运行。

5.2.6.4 测试结论

本次 FreeRTOS 联调结果表明，系统在多任务并发条件下已形成稳定的工程化实现链路：采样任务能够持续输出有效数据，融合与判定逻辑可正确驱动执行器动作，通信任务可稳定完成上/下行协同，异常情况下具备快速恢复能力。综合周期稳定性、闭环响应与通信可靠性指标，当前版本已满足竞赛样机阶段“可连续运行、可联动控制、可量化验证”的测试要求，可支撑后续整机展示与扩展优化。

5.3 算法测试

5.3.1 视觉检测测试

鱼类检测模型在自建的测试集（约 500 帧，覆盖不同光线和鱼的姿态）上做评估，叶片分割模型在单独的叶片测试集（约 200 帧）上评估，结果如表 5-12 所示。

表 5-12 视觉检测测试结果

模型	测试集规模	mAP@0.5	推理延迟（树莓派 5）	备注
鱼类检测（YOLOv8n）	500 帧	87.3%	~48ms	INT8 量化部署
叶片分割（YOLOv8n-seg）	200 帧	82.6%	~65ms	水面反光场景误检率偏高，待优化

5.3.2 机械臂任务成功率测试

对三个自动化任务各进行了 20 次测试，统计任务一次成功（无需人工干预）的比例，结果如表 5-13 所示。

表 5-13 机械臂自动化任务成功率

任务	测试次数	一次成功次数	成功率	主要失败原因
自动喂食	20	18	90%	2 次瓶子被碰歪导致抓取偏移
捞取落叶	20	15	75%	水面反光导致叶片漏检 3 次，抓取姿态偏差 2 次
松土	20	19	95%	1 次滑轨复位偏差导致位置偏移

捞叶成功率相对低一些，主要问题是水面反光干扰，后续计划加偏振滤镜或改用近红外补光来改善。

5.4 软件测试

5.4.1 接口测试

后端启动后，对关键接口做了冒烟测试，结果如表 5-14 所示。

表 5-14 关键接口冒烟测试

测试项	测试方法	预期结果	实际结果
登录接口	默认账号 admin/admin123 调用 /api/login	返回 JWT	成功
传感器接口	携带 JWT 调用 /api/sensors	返回 5 类数据	成功，样例：水温 24.99℃，pH 6.87
机械臂控制	向 /api/robot/control 发 up	坐标+1，WebSocket 推日志	成功，坐标返回 {x:0, y:0, z:1}
模式切换	/api/mode 来回切换 demo/service	状态变化正确	成功
视频流	访问 /api/video/tank	MJPEG 流持续输出	成功，Content-Type 正确

5.4.2 前端交互测试

前端主要页面功能验证情况：控制面板传感器卡片和双路视频集成正常；机械臂控制页三种控制方式（按钮/键盘/摇杆）均可用，预设任务按钮触发后日志面板有反馈；LLM 交互页文字输入和语音输入均可用，大模型回答正常展示；历史数据页本地模拟数据展示和分页功能正常。

第6章 应用前景

6.1 居家消费市场

AquaGarden 最直接的应用场景是城市家庭。随着宠物经济持续升温，国内观赏鱼养殖用户

群体已具备相当规模，但现有产品普遍停留在“传统鱼缸+简单自动化”阶段，用户的核心痛点——维护耗时、异常发现滞后、废水浪费——始终没有被真正解决。AquaGarden 的鱼菜共生自动调度和机械臂多任务执行，可以把用户的日常维护成本从“每天都要操心”降低到“偶尔看一眼 Web 端数据”，对于生活节奏快的上班族和学生群体来说实用性比较突出。

从产品形态来看，系统的“一左一右”双区布局兼顾了观赏性和功能性，放在客厅或书房既是一套生态景观，又能作为实用的室内绿化和鱼类养殖系统。延时摄影、鱼类行为分析等功能也增加了一定的情感价值，让用户对自己“养的东西”有更强的参与感。

中长期来看，随着系统迭代和成本下降，AquaGarden 具备向更小体积、更低价格区间扩展的可能，可以覆盖学生宿舍、小户型公寓等对空间更敏感的场景。

6.2 教育与科普场景

鱼菜共生系统本身就是一个很直观的生态学教具，把水质、植物生长、鱼类行为、传感器数据这些概念都具象化了，比课本上的描述好理解得多。AquaGarden 加入了机械臂、深度学习、大模型等技术模块之后，整套系统的教育属性进一步增强——它几乎覆盖了中学和大学低年级理工科生可能接触到的所有热点方向：物联网、计算机视觉、机器人、人工智能、生态学。

在中小学层面，系统可以作为 STEM/综合实践课程的教具，通过“观察鱼的行为→查看传感器数据→理解水质与生物的关系”这条路径，帮助学生建立对复杂系统的直观认识。在高校层面，AquaGarden 的各个技术模块可以分别对应嵌入式系统、计算机视觉、机器人学等课程的实验对象，具备改造成开放平台的潜力。

6.3 商业展示与公共空间场景

体积适中的鱼菜共生生态缸在商业场景中已经有一定的审美认可度，咖啡馆、酒店大堂、企业前台、科技展馆等场所对这类兼顾观赏性和科技感的展示物有实际需求。AquaGarden 在此基础上引入了机械臂和大模型交互，让整套系统变成了一个可以和访客“说话”、可以实时演示自动化操作的互动装置，展示价值比静态的生态缸高出不少。

这个方向也是 AquaGarden 差异化比较明显的地方：当贝、米家等产品以家用为主，没有往“公共展示”方向延伸；而传统鱼菜共生套件的外观和自动化水平都不足以支撑商业展示的需求。

6.4 养老与康复场景

有研究表明，观察水族生物对缓解焦虑、降低血压有一定效果，在老年人群中尤为明显。但传统鱼缸对老年人来说维护难度较大，一旦没有人定期照料很容易崩溃。AquaGarden 的全自动化特性恰好解决了这个问题：系统可以在几乎不需要人工干预的情况下长期稳定运行，老人只需要偶尔看看，不用担心换水和喂食，还能通过语音和机械臂交互，增加一点趣味性。

在养老院或社区活动中心的公共区域，AquaGarden 可以作为公共观赏装置，同时承担空间绿化（盆栽端）和生态展示（鱼缸端）的功能。

6.5 可持续发展与政策契合性

AquaGarden 的废水循环利用设计与当前的可持续发展政策方向高度契合。农业农村部 2024 年将鱼菜共生技术列为主推技术，“双碳”目标下节水减排的农业技术路线也受到政策层面的鼓励。AquaGarden 的居家版本虽然规模小，但其技术逻辑——用鱼类代谢废物替代化肥、实现水的闭环利用——和规模化农业中推广的理念完全一致，具备从居家产品向社区级、楼宇级扩展的基础。

此外，系统的传感器数据采集和大模型分析能力，未来也有可能接入城市级的生态监测平台，为城市微型生态环境的数字化管理提供一种低成本的数据采集节点方案。

各应用场景的适配情况汇总如表 6-1 所示。

表 6-1 应用场景汇总

应用场景	核心价值点	主要目标群体	落地可行性
居家消费	零维护养鱼+自动浇灌盆栽，节省时间	城市上班族、学生	高
教育科普	覆盖 IoT/AI/机器人/生态多学科的实体教具	中小学、高校	高
商业展示	可互动、可演示的科技感生态装置	酒店、展馆、企业	中
养老康复	无需人工维护，语音交互门槛低	养老院、社区中心	中
可持续农业	节水减肥、废物资源化，契合双碳政策	政府、社区农业项目	中长期

参考文献

- [1] AYON M F I, NAHAR S, RAHMAN A, et al. An IoT-Enabled Smart Aquarium System for Real-Time Water Quality Monitoring and Automated Feeding[J/OL]. arXiv, 2026: arXiv:2601.08484. <https://arxiv.org/abs/2601.08484>
- [2] HSIAO S J, SUNG W T. Building a Fish-Vegetable Coexistence System Based on a Wireless Sensor Network[J]. IEEE Access, 2020, 8: 192131-192146. DOI: 10.1109/ACCESS.2020.3032795.
- [3] AGRAWAL A, MAGANTI P, MAITI R R. Cyber Physical Aquaponic System (CyPhA): a CPS Testbed[J/OL]. arXiv, 2023: arXiv:2304.05132.
- [4] Integrated Aquaponics Through Smart Sensors[J]. Harbin Gongcheng Daxue Xuebao/Journal of Harbin Engineering University, 2024.
- [5] ZAMNURI M A H B, QIU S, RIZALMY M A A B, et al. Integration of IoT in Small-Scale Aquaponics to Enhance Efficiency and Profitability: A Systematic Review[J]. Animals, 2024, 14(17): 2555. DOI: 10.3390/ani14172555.
- [6] Smart Fish Feeding in IoT-based Aquarium: A Comprehensive Approach Integrating Computer Vision, Machine Learning and Sensor Technology[C]//2024 IEEE International Conference. IEEE, 2024. DOI: 10.1109/ICCIT60459.2024.10779718.

-
- [7] JOCHER G, CHAURASIA A, QIU J. Ultralytics YOLOv8[EB/OL]. (2023-01)[2026-04-19]. <https://github.com/ultralytics/ultralytics>
- [8] MACENSKI S, FOOTE T, GERKEY B, et al. Robot Operating System 2: Design, architecture, and uses in the wild[J]. Science Robotics, 2022, 7(66): eabm6074. DOI: 10.1126/scirobotics.abm6074.
- [9] 农业农村部办公厅. 关于推介发布 2024 年农业主导品种主推技术的通知[EB/OL]. (2024-04)[2026-04-19]. http://www.moa.gov.cn/govpublic/KJJYS/202404/t20240419_6453087.htm
- [10] 光鉴科技. Deptrum Aurora 930 系列产品用户手册[Z]. 上海: 光鉴科技, 2024.
- [11] MAHMOUD M M M, DARWISH R, BASSIUNY A M. Development of an economic smart aquaponic system based on IoT[J]. Journal of Engineering Research, 2024, 12(4): 886-894. DOI: 10.1016/j.jer.2023.08.024.